Práctica Integradora Unidad 4: Sistema de Auditoría, Monitorización y Mantenimiento de Bases de Datos en Producción

1. DESCRIPCIÓN GENERAL. OBJETIVOS

Trabajas como Administrador de Bases de Datos (DBA) y SysAdmin en **"Almería E-Market"**, un portal de comercio electrónico en plena expansión. Tras el último periodo de rebajas, el servidor MySQL sufrió severas degradaciones de rendimiento y se detectaron modificaciones no autorizadas en tablas sensibles de configuración y privilegios.

El Director de Tecnología (CTO) te ha encargado diseñar e implementar un **entorno automatizado de mantenimiento preventivo, auditoría de seguridad y análisis de salud de base de datos** utilizando programación nativa del servidor (SQL) combinada con scripts de automatización del sistema operativo (Bash).

2. Arquitectura de Datos de la Práctica

Antes de comenzar, debes simular el ecosistema de la base de datos empresarial ejecutando la siguiente estructura:

```
CREATE DATABASE IF NOT EXISTS almeria_emarket;
USE almeria_emarket;

-- Tabla 1: Tabla transaccional crítica que sufre gran fragmentación
CREATE TABLE pedidos (
id_pedido INT AUTO_INCREMENT PRIMARY KEY,
fecha_pedido TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
cliente_email VARCHAR(100),
total DECIMAL(10,2)
);

-- Tabla 2: Tabla de configuración de sistema (objetivo de seguridad)
CREATE TABLE configuracion_tienda (
parametro VARCHAR(50) PRIMARY KEY,
valor VARCHAR(100),
ultima_modificacion TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE
CURRENT_TIMESTAMP,
modificado_por VARCHAR(100)
);

-- Insertar configuración por defecto
INSERT INTO configuracion_tienda (parametro, valor, modificado_por) VALUES
('MANTENIMIENTO_MODE', 'FALSE', 'root@localhost'),
('DESCUENTO_GLOBAL', '10%', 'root@localhost');
```

3. Requerimientos Técnicos de la Práctica

La práctica se divide en **cuatro retos técnicos integrados**. Debes aportar tanto el código fuente como las explicaciones teóricas de su funcionamiento.

Reto A: El Script de Salud de Sistemas (Bash)

Crea un script de Bash llamado monitor_emarket.sh que realice las siguientes acciones de forma secuencial y desasistida:

1. Compruebe si el servicio de MySQL está activo. Si no lo está, intente levantarlo e informe del estado.
2. Compruebe el almacenamiento del servidor. Si el tamaño físico de la base de datos almeria_emarket supera un límite ficticio (ej. 1 MB para pruebas), invoque de manera automática a un procedimiento almacenado dentro de MySQL que limpie los logs.
3. El script de Bash debe registrar todas sus acciones añadiendo líneas a un archivo de registro del sistema ubicado en /var/log/emarket_mantenimiento.log (puedes simularlo en tu home de usuario si no tienes permisos de superusuario).

Reto B: Auditoría de Cambios de Configuración (Triggers)

Necesitamos bloquear y auditar cualquier intento de modificar la tabla configuracion_tienda.

1. Crea una tabla de logs denominada auditoria_config. Debe registrar: id_log (PK), usuario_sistema, parametro_afectado, valor_antiguo, valor_nuevo, fecha_operacion.
2. Diseña un disparador tg_auditar_config que actúe de manera automática guardando los detalles anteriores cada vez que se realice un cambio de valores (UPDATE).
3. Diseña un disparador tg_prohibir_delete_config que **impida físicamente** que cualquier usuario (incluido root) elimine registros de esta tabla de configuración de tienda, lanzando una señal de error específica (SIGNAL SQLSTATE).

Reto C: Diagnóstico Automatizado de Tablas (Metadatos e Information Schema)

El SGBD pierde rendimiento si existen tablas sin índice primario o con excesivo espacio asignado sin usar.

1. Crea un procedimiento almacenado sp_diagnostico_tablas que realice una consulta analítica contra information_schema.tables y information_schema.table_constraints.
2. El procedimiento debe devolver un informe listando todas las tablas del esquema actual (almeria_emarket) que:

- No tengan clave primaria (PRIMARY KEY) definida.
- O tengan un índice de fragmentación (espacio libre o data_free) superior al 10% del total de la tabla.

Reto D: Archivado de Datos mediante Cursores (SQL)

Para evitar que la tabla pedidos crezca indefinidamente y ralentice los índices de búsqueda, debemos archivar los pedidos que tengan más de un año de antigüedad.

1. Crea una tabla idéntica en estructura llamada historico_pedidos.
2. Diseña un procedimiento almacenado sp_archivar_pedidos_antiguos que use un **CURSOR** para recorrer los pedidos anteriores a una fecha dada.
3. El cursor debe leer fila por fila, insertar cada pedido antiguo en historico_pedidos y, una vez confirmado el volcado, eliminarlo de la tabla original pedidos garantizando la integridad de los datos.

4. CALIFICACIÓN

> ⚠️ **IMPORTANTE:** El retraso en el plazo de entrega a través de la plataforma, la entrega del documento en formatos editables no permitidos (como .docx), capturas con identificación del alumno/a o la omisión de los enlaces de las fuentes documentales utilizadas penalizará gravemente la nota o invalidará la práctica.

| **Criterio** | **Excelente (9-10)** | **Aceptable (5-8)** | **Insuficiente (0-4)** | **Peso** |

| **Reto A: Bash Scripting** | El script se ejecuta sin errores, comprueba el servicio, calcula tamaños dinámicamente y loguea cada evento con formato limpio. | El script funciona pero tiene contraseñas explícitas en texto plano o el cálculo del tamaño de la BD es estático o manual. | El script tiene fallos de sintaxis, no comprueba la salud o no escribe logs. | 2.5 pts |

| **Reto B: Triggers** | Diseña correctamente la auditoría, captura el usuario activo y bloquea operaciones de borrado mediante SQLSTATE correctos. | Implementa los triggers pero no controla bien las variables NEW o OLD o usa excepciones genéricas sin documentar. | No controla eventos, faltan los disparadores o no bloquean la eliminación. | 2.5 pts |

| **Reto C: Diagnóstico Catálogo** | Obtiene con total precisión los datos de fragmentación y claves primarias utilizando JOINs correctos en information_schema. | La consulta al catálogo funciona pero el cálculo de la fragmentación de datos está mal estructurado o no filtra por esquema. | No hace uso de metadatos o la consulta devuelve datos erróneos de almacenamiento. | 2.5 pts |

| **Reto D: Cursor de Archivado** | El cursor procesa e integra el flujo de lectura, inserción y borrado ordenadamente, controlando la excepción NOT FOUND. | El cursor realiza la inserción pero no controla correctamente el borrado o carece del manejador de salida seguro del bucle. | No implementa cursor, utiliza un query simple delete/insert (no se adecúa al requisito técnico del examen). | 2.5 pts |